Azərbaycan Respublikası Elm və Təhsil Nazirliyi

Qarabağ Universiteti

Mühəndislik Fakültəsi

Verilənlərin Strukturu və Alqoritmlər

Mexatronika və robototexnika mühəndisliyi və Kompüter mühəndisliyi ixtisası üzrə bakalavr proqramının tələbələri üçün.

Laboratoriya işi № 2

Tələbənin adı və soyadı: Nicat Orucov

Qrup: KOM25A

Müəllim: Şamil Hümbətov

Tarix: 21 oktyabr 2026

2026

Mövzu: Linked List, Stack və Queue Verilənlər Strukturlarının Tətbiqi, Təhlili və Müqayisəsi

Əsas mənbə: *Mark Allen Weiss, Data Structures and Algorithm Analysis in C++ (4th Edition)*

Giriş

Məqsəd

Bu laboratoriya işinin məqsədi Linked List, Stack və Queue verilənlər strukturlarının nəzəri əsaslarını öyrənmək, onları C++ dilində sıfırdan (STL konteynerlərindən istifadə etmədən) implementasiya etmək, eksperimental sınaqlar aparmaq və nəticələri təhlil etməkdir.

Öyrənmə Nəticələri

Laboratoriya işini uğurla tamamlayan tələbə:

- Linked List, Stack və Queue strukturlarının iş prinsiplərini izah edə bilir;
- Verilənlər strukturlarını STL konteynerlərindən istifadə etmədən implementasiya edə bilir;
- Real həyat problemlərini uyğun verilənlər strukturları vasitəsilə modelləşdirə bilir;
- Eksperimental nəticələri cədvəl və qrafiklər vasitəsilə təqdim edə bilir;
- Teorik və praktik mürəkkəblikləri müqayisə edə bilir;
- Performansa təsir edən sistem parametrlərini müəyyən edə bilir.

Tapşırıq 1

Smart Music Playlist (Linked List)

Real Həyat Ssenarisi

Musiqi tətbiqində (Spotify, Apple Music və s.) istifadəçinin playlisti saxlanılır. İstifadəçi mahnı əlavə edə, silə və playlisti görüntüləyə bilir.

A. Araşdırma

1. Array və Linked List arasında əsas fərqlər hansılardır?

Array elementləri yaddaşda ardıcıl yerləşdirir, buna görə i-ci elementə indeks vasitəsilə O(1) vaxtda çatmaq olur. Lakin ölçüsü əvvəlcədən müəyyən edilir (artırmaq üçün elementlərin yeni yerə köçürülməsi lazımdır) və ortadan element əlavə etmək və ya silmək elementlərin sürüşdürülməsini tələb edir, yəni O(n) vaxt aparır. Linked List-də hər element ayrıca node-dur və növbəti node-un ünvanını saxlayır, node-lar yaddaşda ardıcıl olmaya bilər. Ölçü dinamik dəyişir və mövqeyi məlum olan node-un əlavə edilməsi və ya silinməsi yalnız bir neçə pointer-in dəyişdirilməsi ilə O(1) vaxtda həyata keçirilir. Əvəzində i-ci elementə çatmaq üçün siyahını baştan gəzmək (O(n)) və hər node üçün əlavə pointer yaddaşı ayırmaq lazımdır.

2. Niyə musiqi playlistləri üçün Linked List uyğun seçim hesab oluna bilər?

Playlist tez-tez dəyişir. Mahnılar əlavə olunur silinir və sırası dəyişdirilir, mahnıların sayı isə əvvəlcədən məlum deyil. Linked List-də ölçü dinamik böyüyür. Bundan əlavə playlist əsasən ardıcıl oxunur, yəni əsas əməliyyat təsadüfi indekslə müraciət yox ardıcıl gəzmədir və bu Linked List üçün təbiidir.

3. Singly və Doubly Linked List arasında hansı üstünlüklər və çatışmazlıqlar mövcuddur?

Singly Linked List-də hər node yalnız next göstəricisini saxlayır, buna görə yaddaş azdır və implementasiya sadədir, lakin yalnız irəli istiqamətdə gəzmək olar və node-u silmək üçün ondan əvvəlki node-u ayrıca axtarmaq lazımdır. Doubly Linked List-də əlavə prev göstəricisi var. hər iki istiqamətdə gəzmək tail-dən işləmək və verilmiş node-u əvvəlki node-u axtarmadan silmək mümkündür. Çatışmazlıqlar prev və next olduğu üçün yaddaşda daha çox yer ayrılır.

4. Linked List-də axtarış niyə Array-dən daha yavaş ola bilər?

Linked List-də indeks yoxdur, istənilən elementi tapmaq üçün node-lara baştan ardıcıl keçmək lazımdır. Array-də isə indexi biliriksə elementləri tapmaq daha rahatdır, üstəlik sıralanmış array-də binary search ilə O(log n) vaxtda axtarış mümkündür, Linked List-də isə mümkün deyil

5. Playlistdə mahnıların sırasını dəyişmək üçün hansı struktur daha əlverişlidir?

Linked List, xüsusilə Doubly Linked List daha əlverişlidir. Mahnının yerini dəyişmək üçün node-u köhnə yerindən ayırıb yeni yerə bağlamaq, yəni bir neçə

pointer-i dəyişmək kifayətdir. Array-də isə köhnə və yeni mövqe arasındakı bütün elementlər sürüşdürülməlidir.

B. Proqramlaşdırma

Aşağıdakı funksiyaları dəstəkləyən Playlist sistemi hazırlayın:

Song

- Song ID
- Song Name
- Artist

Əməliyyatlar
Add Song

Delete Song

Search Song

Display Playlist

Count Songs

Məhdudiyyət

Hazır STL list istifadəsi qadağandır.

C. Təcrübə və Eksperiment

Playlist sistemini aşağıdakı ölçülərdə test edin:

100 mahnı

1000 mahnı

10000 mahnı

Aşağıdakı əməliyyatların vaxtını ölçün:

Insert

Search

Delete

D. Analiz və Nəticələrin Şərhi

Ən azı aşağıdakı sualları cavablandırın:

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?
2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?
3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?
4. Linked List-in əsas üstünlüyü nədir?
5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

Nəticələr

Cədvəl 2. Linked List əməliyyatlarının icra müddəti

Element sayı	Insert (ms)	Search (ms)	Delete (ms)
100	0.1116	0.0076	0.0575
1000	0.8047	0.8714	4.9431
10000	12.7502	142.456	480.269

D. Analiz və Nəticələrin Şərhi

1. Element sayı artdıqca hansı əməliyyat daha çox təsirləndi?

Ən çox Search əməliyyatı təsirləndi. n 100-dən 10000-ə (100 dəfə) artanda Search müddəti 0.0076 ms-dən 142.456 ms-ə, yəni təxminən 18,744 dəfə artdı. Delete müddəti 8,353 dəfə (0.0575 ms-dən 480.269 ms-ə), Insert müddəti isə yalnız 114 dəfə (0.1116 ms-dən 12.7502 ms-ə) artdı. Yəni Insert n ilə təxminən mütənasib böyüdü, Search və Delete isə xeyli sürətlə böyüdü.

2. Təcrübə nəticələri Big-O nəzəriyyəsini təsdiqləyirmi?

Qismən təsdiqləyir. **Insert:** orta bir əlavənin müddəti 1.12, 0.805 və 1.28 millisaniye olub, yəni demək olar sabitdir. Bu, sona əlavənin (tail göstəricisi ilə) O(1) olması ilə uyğundur. **Search:** hər axtarış O(n)-dir, n axtarışın cəmi isə təxminən n^2 ilə mütənasibdir. n 100 dəfə artanda müddətin 10^4 dəfə artması gözlənilir, ölçülən artım isə 18,744 dəfədir. Bu, nəzəriyyəyə uyğundur; 10^4-dən bir qədər çox olması böyük siyahıda node-ların yaddaşda səpələnməsi və cache-dən az istifadə olunması ilə izah oluna bilər. n=100 üçün ölçülən 0.0076 ms isə timer həssaslığına yaxın olduğundan dəqiq deyil. **Delete:** burada nəzəriyyə ilə uyğunsuzluq var. Silmə ID-lər daxil edilmə sırası ilə (1-dən n-ə) aparıldığı üçün silinəcək node həmişə siyahının başındadır və hər silmə O(1)-dir, yəni ümumi müddətin təxminən 100 dəfə artması gözlənilirdi. Ölçülən artım isə 8,353 dəfədir. Delete əməliyyatı pointer dəyişməsindən əlavə yaddaşın azad edilməsini də əhatə edir

3. Eyni problemi Array istifadə etməklə həll etsəydiniz nəticələr necə olardı?

Sona əlavə etmə dinamik Array-də O(1) olardı və praktikada Linked List-dən daha sürətli işləyərdi, çünki hər element üçün ayrıca yaddaş ayırmaq lazım gəlmir. Axtarış yenə O(n) olardı. Silmə isə ID-yə görə axtarışdan sonra qalan elementlərin sürüşdürülməsini tələb edir. Bizim testdə hər dəfə ilk element silindiyi üçün hər

silmə O(n) sürüşdürmə olardı və Array burada Linked List-dən daha yavaş ola bilərdi.

4. Linked List-in əsas üstünlüyü nədir?

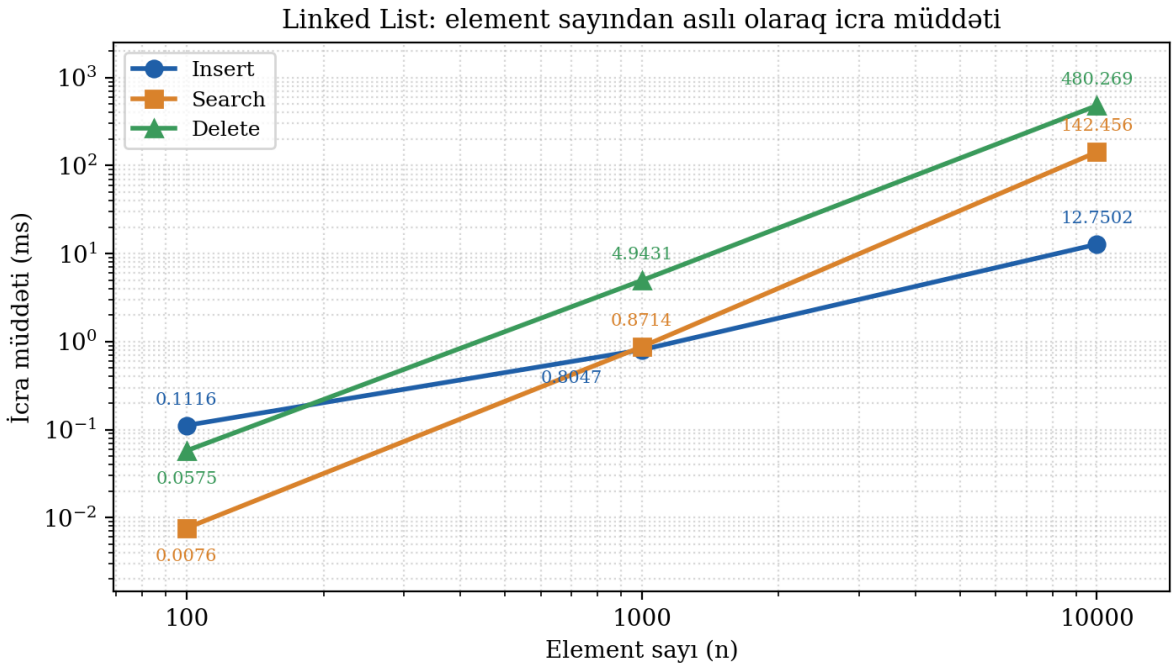
Əsas üstünlük node-un mövqeyi məlum olduqda əlavə və silmənin elementləri sürüşdürmədən, yalnız pointer-ləri dəyişməklə O(1) vaxtda aparılması və ölçünün dinamik olmasıdır (əvvəlcədən ölçü bilmək və böyütmək üçün köçürmə etmək lazım deyil). Bu, eksperimentdə Insert əməliyyatında görünür: 10000 mahnının əlavəsi cəmi 12.7502 ms.

5. Playlist tətbiqində Doubly Linked List istifadə etmək daha məntiqli olardımı?

Bəli, bu tətbiq üçün daha məntiqlidir və implementasiya da buna görə iki tərəfli siyahı kimi aparılmışdır. İstifadəçi həm növbəti, həm də əvvəlki mahnıya keçmək istəyir və bunun üçün prev göstəricisi lazımdır. Ortadakı node-u silmək üçün əvvəlki node-u axtarmaq lazım gəlmir, çünki ona prev vasitəsilə birbaşa çatmaq olur. tail göstəricisi isə sona əlavəni O(1) edir. Əvəzində hər node-da əlavə pointer saxlanılır (64-bit sistemdə 8 bayt) və head, tail, current hallarının ayrıca işlənməsi lazım olur.

Qrafik

Qrafikdə X oxu element sayını, Y oxu icra müddətini göstərir



Şəkil 1. Linked List: Insert, Search və Delete əməliyyatlarının icra müddəti (loqarifmik miqyas)

Tapşırıq 2

Browser History Manager (Stack)

Real Həyat Ssenarisi

Veb brauzerdə istifadəçi müxtəlif səhifələrə keçid edir. “Back” düyməsi basıldıqda əvvəlki səhifələrə qayıdır. Bu mexanizm Stack prinsipi ilə işləyir.

A. Araşdırma

1. Stack strukturunda LIFO prinsipi nədir?

LIFO (Last In, First Out) prinsipinə görə ən son daxil edilən element ilk çıxarılır. Stack-də əlavə etmə (push) və çıxarma (pop) yalnız bir ucdan, yəni top-dan aparılır. Üst-üstə qoyulmuş boşqablar kimi düşünmək olar yalnız ən üstdəki boşqabı götürmək mümkündür.

2. Browser History sistemi nə üçün Stack istifadə edir?

İstifadəçi yeni səhifəyə keçdikdə həmin səhifə stack-ə push olunur. Back düyməsi basıldıqda ən son açılmış səhifə stack-dən çıxarılır (pop) və onun altındakı səhifə yenidən top olur. Geri qayıtma həmişə ən son əməliyyatdan başladığı üçün LIFO bu məsələ ilə təbii uyğun gəlir, üstəlik push və pop O(1) vaxtda işləyir.

3. Undo əməliyyatı necə Stack vasitəsilə həyata keçirilir?

Hər edilən əməliyyat stack-ə push olunur. Undo seçildikdə top-dakı əməliyyat pop edilir və geri qaytarılır. Beləliklə əməliyyatlar tərs ardıcıllıqla, ən sonuncudan başlayaraq ləğv olunur. Redo üçün ləğv edilən əməliyyatlar ikinci bir stack-də saxlanıla bilər.

4. Queue istifadə olunsaydı hansı problemlər yaranardı?

Queue FIFO prinsipi ilə işləyir, yəni Back basıldıqda ən son baxılan səhifəyə yox, ən əvvəl açılmış səhifəyə qayıdılardı. İstifadəçinin hərəkət ardıcıllığı pozulardı və ən son əməliyyatı geri qaytarmaq mümkün olmazdı. Brauzer və Undo kimi sistemlər üçün Queue məntiqi baxımdan yararsızdır.

5. Linked List və Array əsaslı Stack implementasiyalarını müqayisə edin.

Array əsaslı Stack-də elementlər ardıcıl yerləşir, push və pop yalnız indeksi dəyişir, ona görə sürətlidir. Çatışmazlıq ölçünün məhdud olması və ya dolduqda ölçünün böyüdülməsi üçün O(n) köçürmə lazım olmasıdır (amortizə edilmiş O(1)). Linked List əsaslı Stack-də ölçü məhdudiyyəti yoxdur və hər push strikt O(1)-dir, lakin hər element üçün ayrıca yaddaş ayrılır və azad edilir, əlavə pointer yaddaşı lazımdır və yaddaşa müraciət cache baxımından daha az səmərəlidir.

B. Proqramlaşdırma

Aşağıdakı menyunu hazırlayın:

Visit Page

Back

Current Page

Display History

Exit

Məlumat
Page ID

URL

Visit Time

Məhdudiyyət

Hazır STL stack istifadəsi qadağandır.

C. Təcrübə və Eksperiment

Aşağıdakı həcmdə əməliyyatlar yaradın:

100 Push/Pop

1000 Push/Pop

10000 Push/Pop

Ölçün:

Push müddəti

Pop müddəti

Cədvəl

Cədvəl 4. Stack əməliyyatlarının icra müddəti

Əməliyyat sayı	Push (ms)	Pop (ms)
100	0.1972	0.1923
1000	2.0462	16.8935
10000	32.7293	1221.36

D. Analiz və Nəticələrin Şərhi

1. Push və Pop əməliyyatları arasında fərq müşahidə olundumu?

Bəli, n artdıqca fərq çox böyüdü. n=100 üçün müddətlər demək olar eynidir (Push 0.1972 ms, Pop 0.1923 ms). n=1000 üçün Pop Push-dan 8.3 dəfə (16.8935 ms və

2.0462 ms), n=10000 üçün isə 37.3 dəfə yavaş olmuşdur (1221.36 ms və 32.7293 ms).

2. **Əldə olunan nəticələr O(1) mürəkkəbliyini dəstəkləyirmi?**
   Push üçün dəstəkləyir n 100 dəfə artsa da bir əməliyyatın müddəti təxminən sabit qalıb. Pop üçün isə nəticələr O(1)-i dəstəkləmir

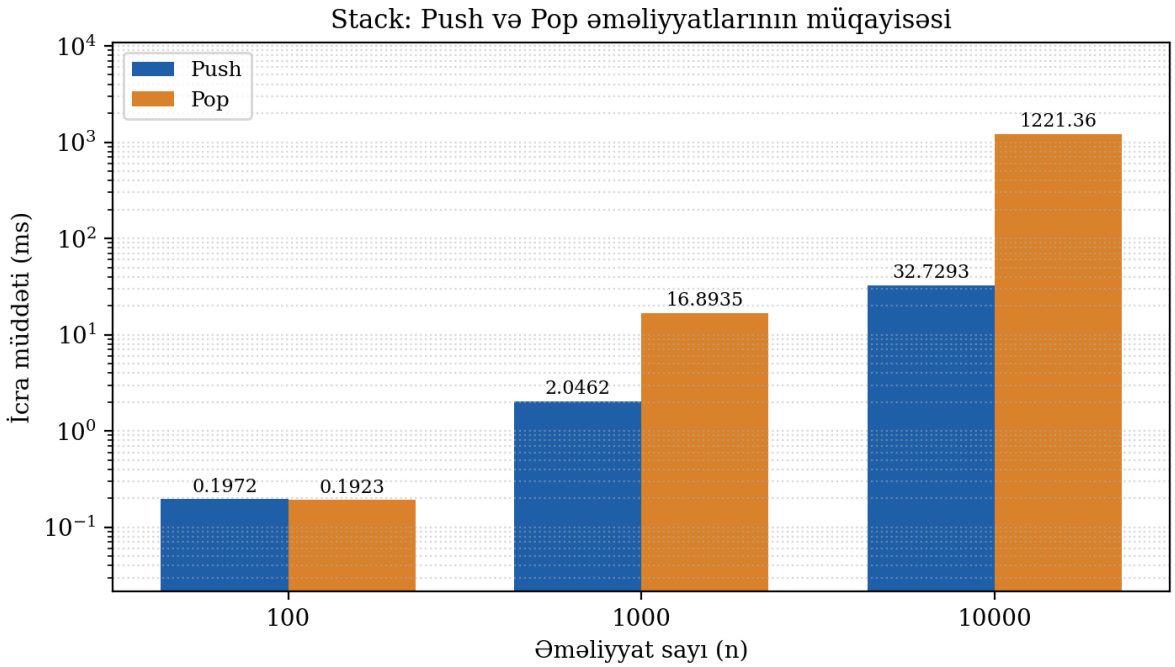
3. **Array əsaslı Stack ilə müqayisədə hansı fərqlər yarana bilər?**
   Array əsaslı Stack-də push və pop yalnız top indeksini dəyişir, hər əməliyyat üçün ayrıca new və delete lazım deyil. Buna görə o, praktikada daha sürətli işləyər və bizim müşahidə etdiyimiz delete ilə bağlı yavaşlama orada baş verməzdi. Elementlər ardıcıl yerləşdiyi üçün cache də daha səmərəli istifadə olunar. Əvəzində Array-in ölçüsü məhduddur və dolduqda böyütmək üçün elementləri yeni yerə köçürmək (O(n)) lazım gəlir. Linked List əsaslı Stack isə heç vaxt "dolmur", lakin hər node üçün əlavə pointer yaddaşı tələb edir.

4. **Brauzerlərdə tarixçənin saxlanması üçün niyə Stack istifadə olunur?**
   Back düyməsi həmişə istifadəçinin ən son baxdığı səhifədən əvvəlki səhifəyə qayıtmalıdır, yəni əməliyyatlar ən sonuncudan başlayaraq geri alınır. Bu, LIFO prinsipinin tam özüdür. Stack-də push və pop O(1) olduğu üçün tarixçə böyük olsa da Back əməliyyatı sürətli qalır.

5. **Stack-in real sistemlərdə başqa hansı tətbiqləri mövcuddur?**
   Stack-in əsas tətbiqləri bunlardır mətn və qrafik redaktorlarında Undo/Redo, proqramlaşdırma dillərində funksiya çağırışları üçün call stack (rekursiya), riyazi ifadələrin qiymətləndirilməsi (infix-dən postfix-ə çevirmə və postfix hesablama), mötərizələrin düzgünlüyünün yoxlanması, qrafda dərinlik üzrə axtarış (DFS) və geriyə qayıtma (backtracking) alqoritmləri, məsələn, labirint həlli.

Qrafik

Push və Pop əməliyyatlarının performans müqayisəsi aşağıda verilmişdir (Y oxu loqarifmik miqyasdadır).



Şəkil 2. Stack: Push və Pop əməliyyatlarının icra müddəti (loqarifmik miqyas)

Tapşırıq 3

Airport Check-in Queue System (Queue)

Real Həyat Ssenarisi

Hava limanında sərnişinlər qeydiyyatdan keçmək üçün növbəyə dayanırlar. Birinci gələn sərnişin birinci xidmət alır (FIFO).

A. Araşdırma

1. Queue strukturunun FIFO prinsipi nədir?

FIFO (First In, First Out) prinsipinə görə ilk daxil olan element ilk çıxarılır. Queue-da yeni elementlər arxadan (rear) əlavə edilir (enqueue), elementlər isə öndən (front) çıxarılır (dequeue). Adi həyatdakı növbə ilə eyni məntiqdir.

2. Queue və Stack arasında əsas fərqlər hansılardır?

Stack LIFO prinsipi ilə işləyir və yalnız bir ucdan (top) istifadə olunur, Queue isə FIFO prinsipi ilə iki ucdan işləyir: əlavə arxadan, çıxarma öndən. Stack ən son əlavə edilən elementi, Queue isə ən əvvəl əlavə edilən elementi qaytarır. Stack Undo, rekursiya və backtracking üçün, Queue isə sıra ilə xidmət, tapşırıqların planlaşdırılması və bufer kimi məsələlər üçün uyğundur.

3. Hava limanında növbə idarəetməsi üçün Queue niyə uyğundur?

Hava limanında ədalətlilik tələb olunur: əvvəl gələn sərnişin əvvəl xidmət almalıdır. Queue elementlərin daxil olma ardıcıllığını avtomatik saxlayır. Əlaqəli siyahıda rear göstəricisi ilə enqueue və front göstəricisi ilə dequeue hər ikisi O(1) vaxtda həyata keçirilir, sərnişin sayı artsa da sistem sürətli qalır.

4. Queue istifadə olunmasa hansı problemlər yarana bilər?

Gəliş ardıcıllığı izlənilmədikdə kimin əvvəl gəldiyi bilinməz və xidmət ədalətsiz olar. Əgər Stack istifadə olunsa, axırıncı gələn sərnişin ilk xidmət alar, əvvəl gələnlər isə uzun müddət gözləməli olar. Array ilə növbə qurulsa və hər dequeue-da qalan elementlər sürüşdürülsə, bu əməliyyat O(n) olar və sərnişin sayı artdıqca sistem yavaşlayar.

5. Circular Queue nə kimi üstünlüklər təqdim edir?

Array əsaslı adi Queue-da dequeue-dan sonra boşalan ön xanalar təkrar istifadə olunmur və ya elementlərin sürüşdürülməsi lazım gəlir. Circular Queue-da front və rear indeksləri massivin sonuna çatdıqda başa qayıdır (modul əməliyyatı ilə), nəticədə boşalan yerlər təkrar istifadə olunur, elementləri sürüşdürmək lazım gəlmir, enqueue və dequeue O(1) qalır və yaddaş sabit ölçülü bufer kimi səmərəli istifadə edilir. Çatışmazlıq ölçünün sabit olması və dolu/boş vəziyyətlərinin ayrıca idarə edilməsidir.

B. Proqramlaşdırma

Aşağıdakı sistemi yaradın:

Passenger
Passenger ID

Name

Flight Number

Əməliyyatlar
Add Passenger

Check-In Passenger

Peek Next Passenger

Display Queue

Count Passengers

C. Təcrübə və Eksperiment

Süni olaraq aşağıdakı ölçülərdə növbələr yaradın:

100 sərnişin

1000 sərnişin

10000 sərnişin

Ölçün:

Enqueue

Dequeue

Peek

Cədvəl

Cədvəl 6. Queue əməliyyatlarının icra müddəti

Element sayı	Enqueue (ms)	Dequeue (ms)	Peek (ms)
100	0.1666	0.0778	0.0003
1000	0.7933	4.4105	0.0008
10000	11.8371	485.251	0.0069

D. Analiz və Nəticələrin Şərhi

1. FIFO prinsipi praktiki nəticələrdə necə müşahidə olunur?
Peek və Check-In həmişə ən əvvəl əlavə edilən sərnişini göstərmiş və çıxarmışdır, Check-In-dən sonra növbədə digerləri qalmışdır. Eksperimentdə isə sərnişinlər 1-

dən n-ə qədər ardıcıl əlavə edilir və dequeue həmişə front-dakı sərnişini çıxarır, yəni çıxış sırası giriş sırası ilə eynidir.

2. **Queue əməliyyatlarının mürəkkəbliyi nəzəri nəticələrə uyğundurmu?**
   Qismən uyğundur. **Enqueue** və O(1) ilə uyğundur. **Peek:** n=10000 üçün 10000 çağırış cəmi 0.0069 ms çəkmişdir (bir çağırış təxminən 0.7 ns), peek heç bir node-u gəzmədiyi üçün sabit müddətlidir. n=100 üçün ölçülən 0.0003 ms timer həssaslığına yaxın olduğundan dəqiq deyil. **Dequeue:** nəzəri olaraq O(1) olmalı idi (front göstəricisi irəli çəkilir və bir node silinir), lakin orta bir dequeue 0.778 µs-dən 4.41 µs-ə, sonra 48.5 µs-ə qədər artmışdır. Bu artım alqoritmin yox, yaddaşın azad edilməsi (delete) zamanı mühitin təsirindən qaynaqlanır

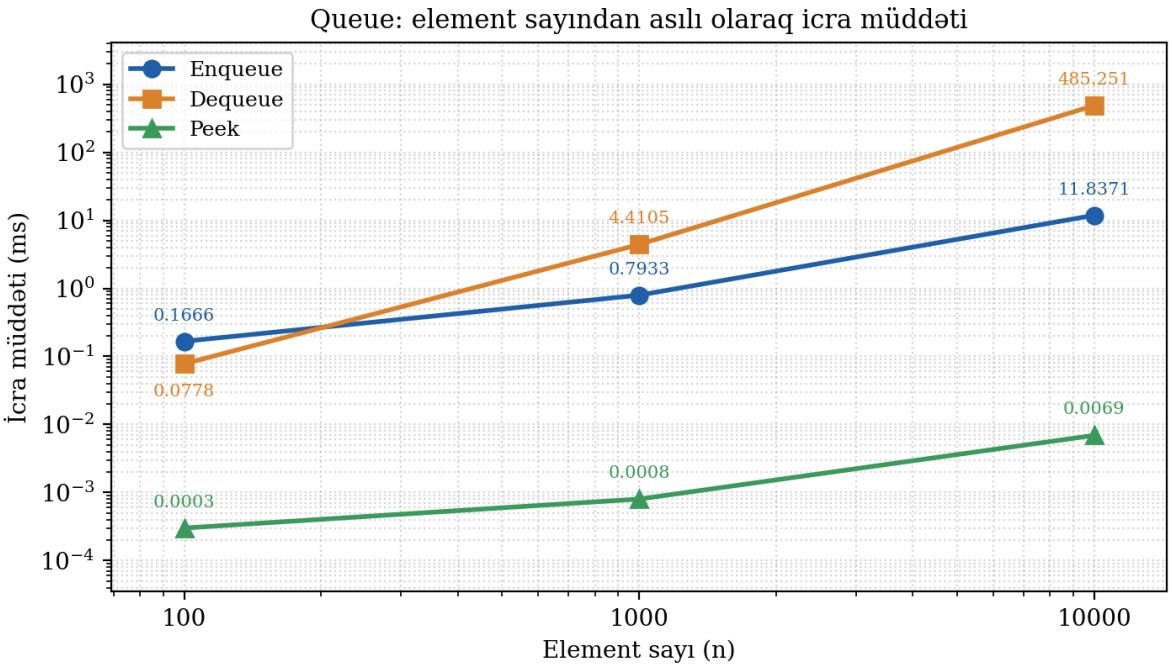
3. **Queue əvəzinə Stack istifadə olunsa nə baş verərdi?**
   Stack LIFO prinsipi ilə işlədiyi üçün axırıncı gələn sərnişin ilk xidmət alardı, əvvəl gələnlər isə növbənin sonuna keçmiş olardı.

4. **Növbə ölçüsü artdıqca hansı dəyişikliklər müşahidə olundu?**
   n 100-dən 10000-ə artanda Enqueue ümumi müddəti 0.1666 ms-dən 11.8371 ms-ə, yəni təxminən 71 dəfə artdı və n ilə təxminən mütənasib qaldı. Peek ümumi müddəti çox kiçik qaldı (0.0003 ms-dən 0.0069 ms-ə). Dequeue ümumi müddəti 0.0778 ms-dən 485.251 ms-ə, yəni təxminən 6,237 dəfə artdı. Bu, n-in artımından (100 dəfə) xeyli çoxdur və Dequeue-nun orta müddətinin böyüməsini göstərir (bir dequeue 62 dəfə artıb). Ən böyük ölçüdə Dequeue ümumi müddətin demək olar ki, hamısını təşkil etmişdir: 485.251 ms 497.095 ms-dən.

5. **Circular Queue istifadə etməyin üstünlükləri nələrdir?**
   Circular Queue Array əsaslı növbədə yaddaşı səmərəli istifadə etməyə imkan verir: dequeue-dan sonra boşalan xanalar yenidən istifadə olunur və elementləri sürüşdürmək lazım gəlmir, enqueue və dequeue O(1) qalır. Hər sərnişin üçün ayrıca new və delete əvəzinə əvvəlcədən ayrılmış bir yaddaş bloku istifadə olunduğundan bu tapşırıqda müşahidə etdiyimiz delete ilə bağlı xərc də aradan qalxardı. Hava limanındakı kimi maksimum tutumu məlum olan növbələr üçün uyğundur. Çatışmazlıq ölçünün sabit olması və növbənin dolu və boş olduğu halların ayrıca idarə olunmasıdır.

Qrafik

Element sayı ilə əməliyyat vaxtları arasındakı əlaqə aşağıdakı qrafikdə göstərilmişdir (hər iki ox loqarifmik miqyasdadır).



Şəkil 3. Queue: Enqueue, Dequeue və Peek əməliyyatlarının icra müddəti (loqarifmik miqyas)

Sistem Məlumatları

Ölçmələr aşağıdakı kompüterdə aparılmışdır.

Cədvəl 7. Test kompüterinin parametrləri

Parametr	Dəyər
Computer Model	Lenovo LOG 15IRX9
CPU	Intel Core i5-13450HX
RAM	24GB
GPU	NVIDIA GeForce RTX 4050 (6 GB)
Operating System	Windows 11 Pro (25H2)
Compiler	g++ (GCC)
Compiler Version	16.2.0
IDE	Visual Studio Code
Screen Resolution	1920x1080
Storage Type (SSD/HDD)	SSD
Power Mode	Performance
Other Factors	

Sistem parametrlərinin nəticələrə təsiri

CPU. Intel Core i5-13450HX hibrid arxitekturalı prosessordur (performans və səmərəlilik nüvələri). Proqram tək thread-də işlədiyi üçün hansı nüvədə və hansı tezlikdə işləməsi ölçmələrə təsir edir. Prosessor tezliyi yükə görə dinamik dəyişdiyindən kiçik n üçün (mikrosaniyələr səviyyəsində) ölçülən müddətlərdə nisbətən böyük sapma mümkündür.

RAM və cache. Test həcmi kiçikdir (10000 node), ona görə RAM-ın həcmi və sürəti əsas faktor deyil.

GPU. Proqramlar GPU-dan istifadə etmir, ona görə RTX 4050-nin ölçmələrə təsiri yoxdur.

Kompilyator və IDE. g++ ilə kompilyasiyada optimizasiya səviyyəsi (-O0, -O2) ölçülən müddətləri əhəmiyyətli dərəcədə dəyişir..

Yaddaş (SSD/HDD) və ekran. Eksperimentlərdə disk əməliyyatı və ekran çıxışı vaxt ölçülən hissəyə daxil deyil, ona görə SSD və ekran ayırdetməsinin təsiri yoxdur.

Enerji rejimi. Performance rejimində prosessor yüksək tezlikdə işləyir və tezliyin enerji qənaəti üçün azaldılması ehtimalı azdır. Enerji qənaəti rejimində müddətlərin artması gözlənilərdi.

Nəticə

Bu laboratoriya işində Linked List, Stack və Queue strukturları C++ dilində, STL konteynerlərindən istifadə etmədən, öz Node və idarəedici siniflərimlə implementasiya etdim və hər biri real həyat ssenarisi (musiqi playlisti, brauzer tarixçəsi, hava limanı növbəsi) üzərində sınaqdan keçirdim.

Eksperiment nəticələrinə görə əlavə etmə əməliyyatlarında (Insert, Push, Enqueue) orta bir əməliyyatın müddəti n artdıqca təxminən sabit qaldı (təxminən 0.793–3.27 µs), bu da O(1) nəzəriyyəsini təsdiqləyir. Linked List-də ardıcıl axtarış n artdıqca təxminən n^2 ilə mütənasib böyüdü: n 100 dəfə artanda müddət 18,744 dəfə artdı, bu da hər axtarışın O(n) olması ilə uyğundur. Peek əməliyyatı praktiki olaraq sabit müddətli oldu.

Delete, Pop və Dequeue əməliyyatları nəzəri olaraq O(1) olsa da, ölçmələrdə n artdıqca orta müddət kəskin artdı: məsələn, orta bir Pop 1.92 ms-dən 122 ms-ə qədər artdı. Bu fərq alqoritmin özündən yox, yaddaşın azad edilməsi zamanı mühitin təsirindən qaynaqlanır

Məhdudiyyət: hər ölçü bir dəfə işə salınmışdır, buna görə nəticələr mütləq dəyərlər kimi yox, tendensiya kimi qiymətləndirilməlidir. Ümumi müqayisə aşağıdakı cədvəldə verilmişdir.

Cədvəl 8. n = 10000 üçün əməliyyatların ümumi müqayisəsi

Struktur	Əməliyyat	Ümumi (ms)	Orta (µs)	Nəzəri	Müşahidə
Linked List	Insert	12.7502	1.28	O(1)	Uyğundur
Linked List	Search	142.456	14.2	O(n)	Uyğundur
Linked List	Delete	480.269	48	O(1)*	Uyğun deyil
Stack	Push	32.7293	3.27	O(1)	Uyğundur
Stack	Pop	1221.36	122	O(1)	Uyğun deyil
Queue	Enqueue	11.8371	1.18	O(1)	Uyğundur
Queue	Dequeue	485.251	48.5	O(1)	Uyğun deyil
Queue	Peek	0.0069	0.00069	O(1)	Uyğundur

** Test zamanı silinən node həmişə siyahının başında olduğu üçün hər silmə O(1)-dir.*

İstinadlar

[1] Mark Allen Weiss. Data Structures and Algorithm Analysis in C++ (4th Edition). Pearson. Chapter 3: Lists, Stacks, and Queues.

[2] cppreference.com. Date and time utilities: std::chrono. https://en.cppreference.com/w/cpp/chrono

AI Usage Declaration

Tool: Google Gemini

Purpose: C++ <chrono> kitabxanasının öyrənilməsi və kodun icra müddətinin ölçülməsi texnikasının başa düşülməsi, kodunu işləmə zamanının uyğun gəlməməsiylə bağlı araşdırma, pdf faylında qrafiklərin hazırlanması

Used Sections: C. Təcrübə və Eksperiment (Tapşırıq 1, 2 və 3 — icra müddətinin ölçülməsi hissəsi).